// for evaluation of postfix

```
#include <iostream>
using namespace std;

class Evalution
{
public:
  char *post, ch;
  int opstk[10], result, top, i;

  Evalution()
  {
    post = new char[20];
    top = -1;
    i = 0;
  }

  void getdata()
  {
    cout << "\nEnter Postfix expression: ";
    cin >> post;
  }

  void Eval()
  {
    while (post[i] != '\0')
    {
      if ((post[i] >= 'A' && post[i] <= 'Z') ||
        (post[i] >= 'a' && post[i] <= 'z'))
      {
        cout << "\nEnter Value of " << post[i] << " = ";
        cin >> opstk[++top];
      }
      else
      {
        int x, y, z;
        x = opstk[top--];
        y = opstk[top--];

        switch (post[i])
        {
        case '+':
          z = y + x;
          break;
```

```
            case '-':
              z = y - x;
              break;

            case '*':
              z = y * x;
              break;

            case '/':
              z = y / x;
              break;

            case '^':
              z = pow(y, x);
              break;
            }

            opstk[++top] = z;
          }

          i++;
        }

        cout << "\nThe result is: " << opstk[top];
    }
};

int main()
{
    Evalution e;

    e.getdata();
    e.Eval();

    return 0;
}
```

//converting infix expression to postfix

```
#include <iostream>
using namespace std;

class InToPo
{
  char *p, *q, *s;
  int top;
public:
  InToPo()
  {
    p = new char[50];
    q = new char[50];
    s = new char[50];
    top = -1;
  }
  void getInfix()
  {
    cout << "\nEnter the infix Expression: ";
    cin >> q;
  }
  int precedence(char ch)
  {
    switch (ch)
    {
    case '^':
      return 4;

    case '*':
    case '/':
      return 3;

    case '+':
    case '-':
      return 2;

    default:
      return 1;
    }
  }
  void convert()
  {
    int i = 0, j = 0;
    while (q[i] != '\0')
    {
```

```
  if ((q[i] >= 'a' && q[i] <= 'z') ||
     (q[i] >= 'A' && q[i] <= 'Z'))
  {
     p[j++] = q[i];
  }
  else if (q[i] == '(')
  {
     s[++top] = '(';
  }


  else if (q[i] == '+' || q[i] == '-' ||
         q[i] == '/' || q[i] == '*' ||
         q[i] == '^')
  {
     while (top != -1 &&
          s[top] != '(' &&
          precedence(s[top]) >= precedence(q[i]))
     {
        p[j++] = s[top--];
     }

     s[++top] = q[i];
  }
  else if (q[i] == ')')
  {
     while (top != -1 && s[top] != '(')
     {
        p[j++] = s[top--];
     }

     if (top != -1)
        top--;
  }

  i++;
}
while (top != -1)
{
  p[j++] = s[top--];
}

p[j] = '\0';
```

```
        cout << "\nPostfix is: " << p;
    }
};

int main()
{
    InToPo obj;

    obj.getInfix();
    obj.convert();

    return 0;
}
```

//circular queue

```
#include <iostream>
using namespace std;

class queue
{
private:
   int a[10], f, r, item;

public:
   queue()
   {
      f = -1;
      r = -1;
   }

   void insert();
   void del();
   void show();
   int isempty();
};

int queue::isempty()
{
   if (f == -1)
      return (1);
   else
      return (0);
}

void queue::insert()
{
   if ((f == 0 && r == 9) || (f == r + 1))
   {
      cout << "\n queue overflow:";
   }
   else
   {
      cout << "\nenter the item:";
      cin >> item;

      if (isempty())
      {
         f = r = 0;
```

```
    }
    else
    {
      if (r == 9)
        r = 0;
      else
        r++;
    }
    a[r] = item;
  }
}

void queue::del()
{
  if (isempty())
  {
    cout << "\n queue underflow:";
  }
  else
  {
    item = a[f];

    if (f == r)
      f = r = -1;
    else if (f == 9)
      f = 0;
    else
      f++;

    cout << "\nDelete item is:" << item;
  }
}

void queue::show()
{
  if (isempty())
  {
    cout << "\n queue underflow:";
  }
  else if (f <= r)
  {
    for (int i = f; i <= r; i++)
      cout << "\t" << a[i];
  }
}
```

```
int main()
{
   queue v;
   int ch;

   do
   {
      cout << "\n1.insert\n2.delete\n3.show\n4.exit";
      cout << "\nenter your choice:";
      cin >> ch;

      switch (ch)
      {
      case 1:
         v.insert();
         break;
      case 2:
         v.del();
         break;
      case 3:
         v.show();
         break;
      case 4:
         break;
      default:
         cout << "\nInvaild choice:";
      }
   } while (ch != 4);

   return 0;
}
```

//Simple Queue

```
#include <iostream>
using namespace std;

class queue
{
  int a[4], R, F;

public:
  queue()
  {
    F = R = 0;
  }

  void Insert();
  void Delete();
  void Show();
};

void queue::Insert()
{
  int item;
  if (R >= 3)
  {
    cout << "\n overflow";
  }
  else
  {
    cout << "\n enter item:";
    cin >> item;
    R++;
    a[R] = item;
    if (F == 0)
    {
      F = 1;
    }
  }
}

void queue::Delete()
{
  if (F == 0)
  {
    cout << "\n underflow";
  }
```

```
  else
  {
    cout << "\n Deleted element is :" << a[F];
    if (F == R)
    {
      F = R = 0;
    }
    else
    {
      F++;
    }
  }
}

void queue::Show()
{
  if (F == 0)
  {
    cout << "\n empty";
  }
  else
  {
    cout << "\nelement are:\n";
    for (int i = F; i <= R; i++)
    {
      cout << "\n" << a[i];
    }
  }
}

int main()
{
  queue s;
  int ch;

  cout << "\n";
  cout << "\n 1.INSERT";
  cout << "\n 2.DELETE";
  cout << "\n 3.SHOW";
  cout << "\n 4.EXIT";
  cout << "\n";

  do
  {
    cout << "\n enter choice:";
```

```
        cin >> ch;

        switch (ch)
        {
        case 1:
          s.Insert();
          break;
        case 2:
          s.Delete();
          break;
        case 3:
          s.Show();
          break;
        default:
          cout << "\n wrong choice";
        }
    } while (ch <= 3);

    return 0;
}
```

// linked list

```
#include <iostream>
using namespace std;

class Node
{
public:
  int info;
  Node *next, *prev;
};

class Dlink
{
  Node *first;
  Node *last;
  Node *p, *t;
  Node *n;

public:
  Dlink()
  {
    first = NULL;
    last = NULL;
    p = t = n = NULL;
  }

  void Insert();
  void Delete();
  void Search();
  void DispF();
  void DispB();
  int count();
};

void Dlink::DispF()
{
  if (first == NULL)
    cout << "\nList is Empty";
  else
  {
    cout << "\nList in forward order is: ";

    p = first;

    while (p != NULL)
```

```
    {
      cout << "\t" << p->info;
      p = p->next;
    }
  }
}

// Displays List in Backward Direction
void Dlink::DispB()
{
  if (first == NULL)
    cout << "\nList is Empty";
  else
  {
    cout << "\nReverse List is: ";

    p = last;

    while (p != NULL)
    {
      cout << "\t" << p->info;
      p = p->prev;
    }
  }
}


int Dlink::count()
{
  int cnt = 0;

  p = first;

  while (p != NULL)
  {
    cnt++;
    p = p->next;
  }

  return cnt;
}

void Dlink::Search()
{
  if (first == NULL)
```

```
  {
    cout << "\nList is Underflow";
  }
  else
  {
    int item, loc = 1;

    cout << "\nEnter the item to be searched: ";
    cin >> item;

    p = first;

    while (p != NULL && p->info != item)
    {
      loc++;
      p = p->next;
    }


    if (p == NULL)
    {
      cout << "\nItem is not present in the List";
    }
    else
    {
      cout << "\nThe item " << item
          << " found in the list at location: " << loc;
    }
  }
}

// Insert node
void Dlink::Insert()
{
  int pos;

  cout << "\nEnter the position: ";
  cin >> pos;

  if (pos > 0 && pos <= count() + 1)
  {
    n = new Node();

    cout << "\nEnter the Item: ";
    cin >> n->info;
```

```
    n->next = NULL;
    n->prev = NULL;


    if (first == NULL)
    {
      first = last = n;
    }
    else
    {

      if (pos == 1)
      {
        n->next = first;
        first->prev = n;
        first = n;
      }


      else if (pos == count() + 1)
      {
        last->next = n;
        n->prev = last;
        last = n;
      }


      else
      {
        p = first;

        for (int i = 1; i < pos - 1; i++)
          p = p->next;

        t = p->next;

        p->next = n;
        n->prev = p;

        n->next = t;
        t->prev = n;
      }
    }
}
```

```
  else
  {
    cout << "\nPlease enter valid position";
  }
}

void Dlink::Delete()
{
  if (first == NULL)
  {
    cout << "\nList is UNDERFLOW";
  }
  else
  {
    int pos;

    cout << "\nEnter the position: ";
    cin >> pos;

    if (pos > 0 && pos <= count())
    {

      if (first == last)
      {
        n = first;
        first = last = NULL;
        delete n;
      }


      else if (pos == 1)
      {
        n = first;
        first = first->next;
        first->prev = NULL;
        delete n;
      }


      else if (pos == count())
      {
        n = last;
        last = last->prev;
        last->next = NULL;
        delete n;
```

```
        }


        else
        {
          p = first;

          for (int i = 1; i < pos; i++)
            p = p->next;

          n = p;

          p->prev->next = p->next;
          p->next->prev = p->prev;

          delete n;
        }
      }
      else
      {
        cout << "\nPlease enter valid position";
      }
    }
}

int main()
{
  Dlink p;
  int ch;

  do
  {
    cout << "\n\nMENU:";
    cout << "\n1. Insert";
    cout << "\n2. Delete";
    cout << "\n3. Search";
    cout << "\n4. Display Forward";
    cout << "\n5. Display Backward";
    cout << "\n6. Exit";

    cout << "\nEnter choice: ";
    cin >> ch;

    switch (ch)
    {
```

```
      case 1:
        p.Insert();
        break;

      case 2:
        p.Delete();
        break;

      case 3:
        p.Search();
        break;

      case 4:
        p.DispF();
        break;

      case 5:
        p.DispB();
        break;

      case 6:
        cout << "\nExiting...";
        break;

      default:
        cout << "\nSorry.. Invalid choice";
      }

   } while (ch != 6);

   return 0;
}
```

//In order pre order

```
#include<iostream>
using namespace std;

class node
{
  int info;
  node *left, *right;
  friend class BST;
};

node *root = NULL;

class BST
{
  node *pp, *p, *n, *t, *ptr;
public:
  BST ()
  {
    root = NULL;
    ptr = NULL;
    pp = NULL;
  }

  void create();
  void search();
  void deletion();
  void display (node *, int);
};

void BST::create()
{
  char ch;
  do
  {
    n = new node;
    cout << "\nEnter The Element:";
    cin >> n->info;
    n->left = NULL;
    n->right = NULL;
    if (root == NULL)
      root = n;
    else
    {
      t = root;
```

```
        while (t != NULL)
        {
          p = t;
          if (n->info < t->info)
            t = t->left;
          else
            t = t->right;
        }
        if (n->info < p->info)
          p->left = n;
        else
          p->right = n;
      }
      cout << "\nDo you want to add another element? (y/n): ";
      cin >> ch;
    } while (ch == 'Y' || ch == 'y');
}

void BST::search()
{
    int item, c=0;

    cout << "\n Enter The Item To Be search";
    cin >> item;

    p = root;

    while (p != NULL && item != p->info)
    {
      if (item < p->info)
      {
        p = p->left;
        c++;
      }
      else
      {
        p = p->right;
        c++;
      }
    }

    cout << "\n Item Is found At Level:" << c+1;

    if (p == NULL)
      cout << "\n Item Is Not found.";
```

```
}

void BST::deletion()
{
   int item;
   cout << "\n Enter The Item for Deletion:";
   cin >> item;

   p = root;

   while (p != NULL && p->info != item)
   {
      pp = p;

      if (item < p->info)
         p = p->left;
      else
         p = p->right;
   }

   if (p == NULL)
   {
      cout << "\n Item is Not found in The List.";
      return;
   }

   cout << "\n Deleted Item Is:" << p->info;

   if (p->left != NULL && p->right != NULL)
   {
      node *s = p->left;
      node *ps = p;

      {
         ps = s;
         s = s->right;
      }

      p->info = s->info;
      p = s;
      pp = ps;
   }
   node * c;
   if (p->left != NULL)
      c = p->left;
```

```
    else
      c = p->right;
    if (p == root)
      root = c;
    else
    {
      if (p == pp->left)
        pp->left = c;
      else
        pp->right = c;
    }
    delete p;
}
void BST::display(node *ptr, int level){
    if(ptr != NULL)
    {
      display (ptr->right, level+1);
      cout << "\n";
      if (ptr == root)
        cout << "Root ->";
      else
      {
        for (int i=0; i<level; i++)
          cout << "  ";
      }
      cout << ptr->info;
      display (ptr->left, level+1);
    }
}

int main()
{
    int ch;
    BST b;
    do
    {
      cout << "\n ** MENU *";
      cout << "\n1.Create \n2.Search \n3.Deletion \n4.Display \n5.Exit";
      cout << "\n Enter your choice:";
      cin >> ch;

      switch (ch)
      {
        case 1: b.create(); break;
        case 2: b.search(); break;
```

```
        case 3: b.deletion(); break;
        case 4: b.display (root, 1); break;
        case 5: break;
        default: cout << "\n Wrong choice:";
      }
   } while (ch != 5);

   return 0;
}
```

// Tree traverse of binary search tree

```
#include<iostream>
using namespace std;

class node
{
public:
  int info;
  node* left;
  node* right;
}*root;

class BST
{
public:
  void create();
  void inorder(node*);
  void preorder(node*);
  void postorder(node*);
  void display(node*,int);

  BST()
  {
    root=NULL;
  }
};

void BST::create()
{
  char ch;
  node*n,*t,*p;

  do
  {
    n=new node;
    cout<<"\nEnter the element:";
    cin>>n->info;

    n->left=NULL;
    n->right=NULL;

    if(root==NULL)
      root=n;
    else
    {
```

```
        t=root;

        while(t!=NULL)
        {
          p=t;

          if(n->info<t->info)
            t=t->left;
          else
            t=t->right;
        }

        if(n->info<p->info)
        {
          p->left=n;
          cout<<"Node is added as left sub tree:";
        }
        else
        {
          p->right=n;
          cout<<"Node is added as right sub tree:";
        }
      }

      cout<<"\nDo you want to add another node in the BST?(Y/N):";
      cin>>ch;

   }while(ch=='Y'||ch=='y');
}

void BST::inorder(node*ptr)
{
   if(root==NULL)
   {
      cout<<"Tree is empty"<<endl;
      return;
   }

   if(ptr!=NULL)
   {
      inorder(ptr->left);
      cout<<ptr->info<<" ";
      inorder(ptr->right);
   }
}
```

```
void BST::preorder(node*ptr)
{
  if(root==NULL)
  {
    cout<<"Tree is empty"<<endl;
    return;
  }

  if(ptr!=NULL)
  {
    cout<<ptr->info<<" ";
    preorder(ptr->left);
    preorder(ptr->right);
  }
}

void BST::postorder(node*ptr)
{
  if(root==NULL)
  {
    cout<<"Tree is empty"<<endl;
    return;
  }

  if(ptr!=NULL)
  {
    postorder(ptr->left);
    postorder(ptr->right);
    cout<<ptr->info<<" ";
  }
}

void BST::display(node*ptr,int level)
{
  if(ptr==NULL)
    return;

  display(ptr->right,level+1);

  for(int i=0;i<level;i++)
    cout<<"\t";

  cout<<ptr->info<<endl;
```

```
    display(ptr->left,level+1);
}

int main()
{
    BST b;
    int ch;

    do
    {
        cout<<"\n1.Create ";
        cout<<"\n2.Inorder";
        cout<<"\n3.Preorder";
        cout<<"\n4.Postorder";
        cout<<"\n5.Display Tree Structure";
        cout<<"\n6.Exit";

        cout<<"\nEnter your choice:";
        cin>>ch;

        switch(ch)
        {
            case 1:
                b.create();
                break;

            case 2:
                b.inorder(root);
                break;

            case 3:
                b.preorder(root);
                break;

            case 4:
                b.postorder(root);
                break;

            case 5:
                b.display(root,0);
                break;

            case 6:
                break;
```

```
        default:
           cout<<"Invalid choice";
      }

   }while(ch!=6);

   return 0;
}
```